Deployment Documentation

This documentation outlines the steps required to deploy a Flask server connected to a PostgreSQL database and integrate it with a Flutter application. The deployment process includes creating a PostgreSQL database, connecting it to the Flask server, deploying the server, and updating the API URLs in the Flutter code.

Prerequisites

Before you begin, ensure that you have the following:

- A PostgreSQL database server set up and running.
- A Flask server application that connects to the PostgreSQL database.
- A Flutter application where you want to use the APIs provided by the Flask server.

Step 1: Create a PostgreSQL Database

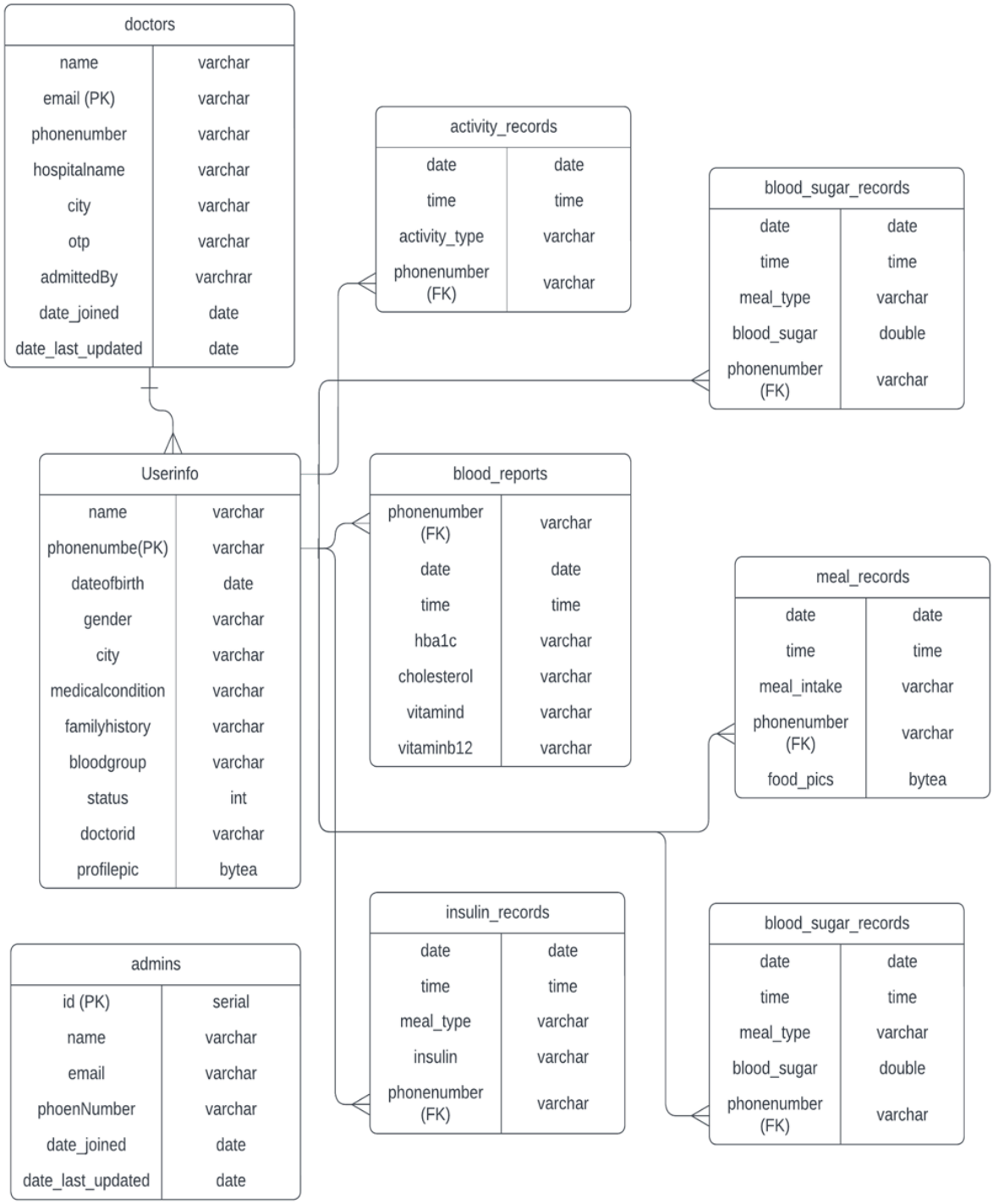
1. Access your PostgreSQL server using an appropriate client or command-line tool.
2. Create a new database for your application.
3. Create the necessary tables and schema within your database for your application data.
4. Make note of the following details:
    a. PostgreSQL Hostname or IP Address
    b. Database Name
    c. Username and Password (if required)

Step 2: Connect PostgreSQL with Flask Server

Ensure that your Flask server is configured to connect to the PostgreSQL database using the information gathered in the previous step.

```
import psycopg2

db_config = {
    'dbname': 'your_database_name',   # Replace with your database name
    'user': 'your_username',          # Replace with your PostgreSQL username
    'password': 'your_password',      # Replace with your PostgreSQL password
    'host': 'your_hostname',          # Replace with your PostgreSQL hostname or IP address
```

```
    'port': 'your_port_number'        # Replace with your PostgreSQL port number (usually 5432)
}
```



Step 3: Deploy the Flask Server

Deploy your Flask server to a production environment using an appropriate web server or deployment platform. Common choices include:

Deploying on a cloud service like AWS, GCP, or Heroku. Using a web server like Nginx or Apache in conjunction with a WSGI server like Gunicorn. Ensure your server is properly configured, and the required dependencies are installed. Refer to the documentation of your chosen deployment method for detailed instructions.

Step 4: Update API URLs in Flutter Code

In your Flutter application, update the API URLs to point to your deployed Flask server. Look for the sections of your code where API requests are made and replace the URLs with the production server's URL.

Example in Flutter using the http package:

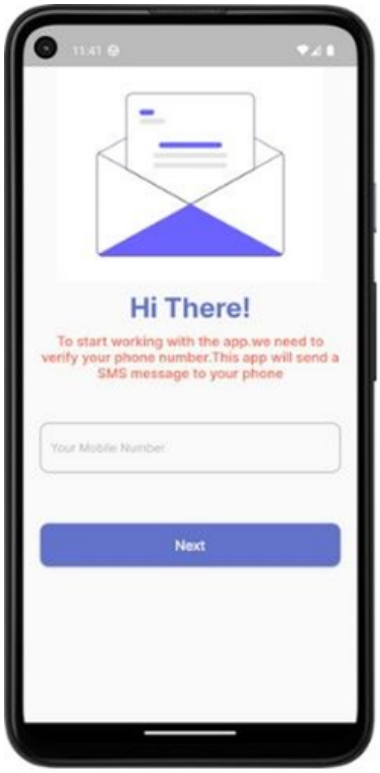
```
import 'package:http/http.dart' as http;

final baseUrl = 'https://your-production-server.com'; // Update with your server's URL

Future<void> fetchData() async {
  final response = await http.get('$baseUrl/api/data');
  // Handle the response data here
}
```

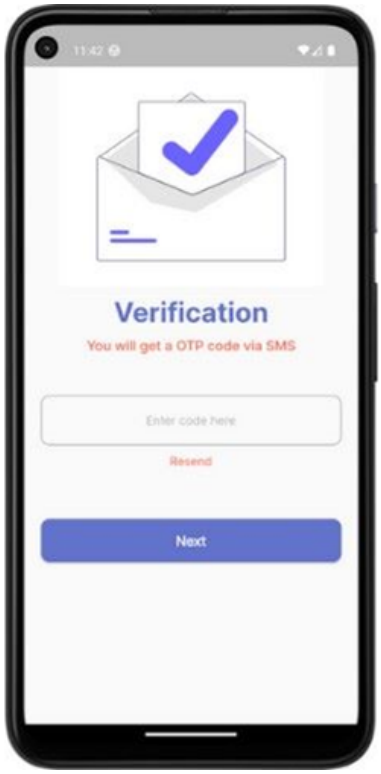
Make sure to replace 'https://your-production-server.com' with the actual URL of your deployed Flask server.

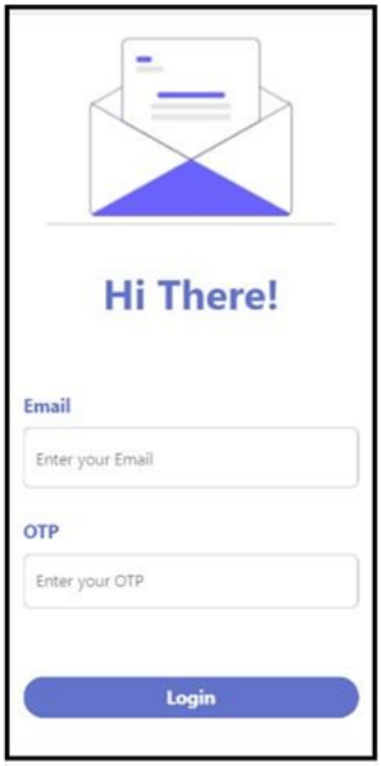
Screenshot

Login Pages

These login pages serve as the initial gateways to the app. Users are required to input their credentials, which will then determine the specific interface they are directed to based on their role within the app.

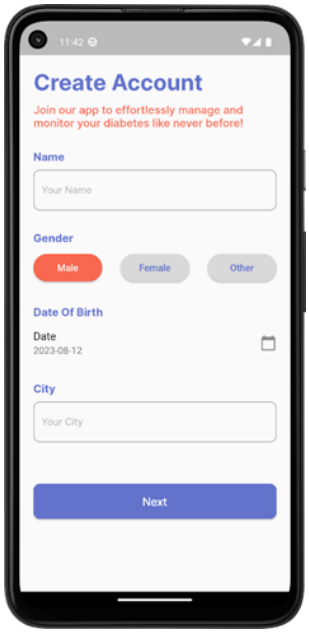


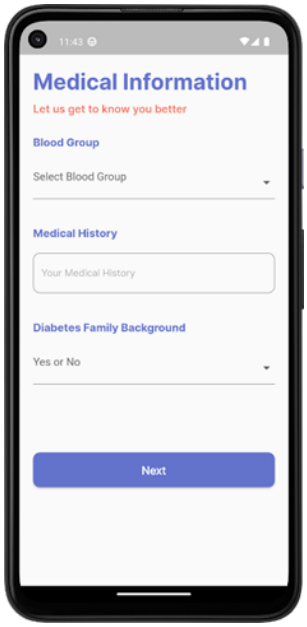


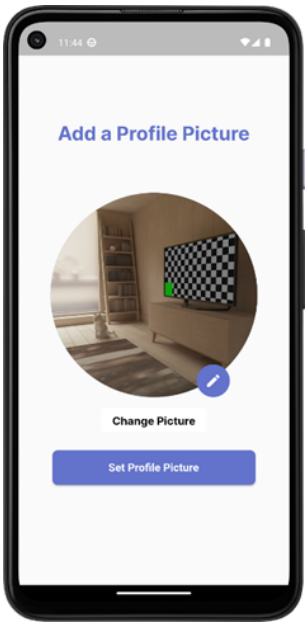


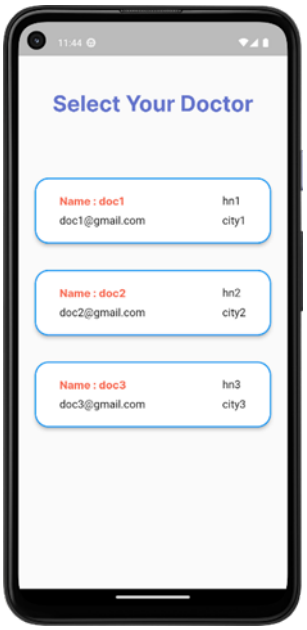
Onboarding Pages

These are the pages which are shown to the patients when the user joins the patients for the first time.



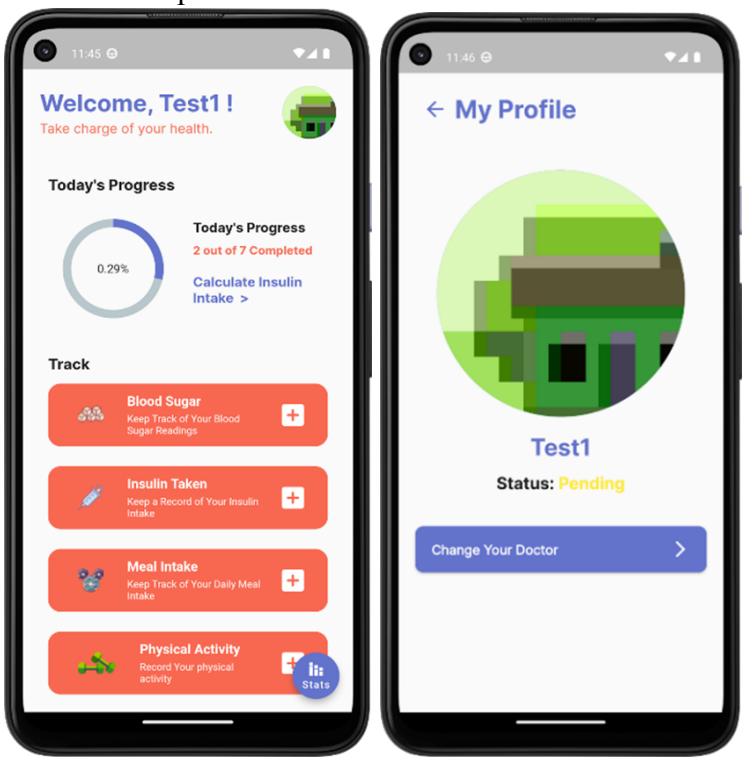






Home and Profile page

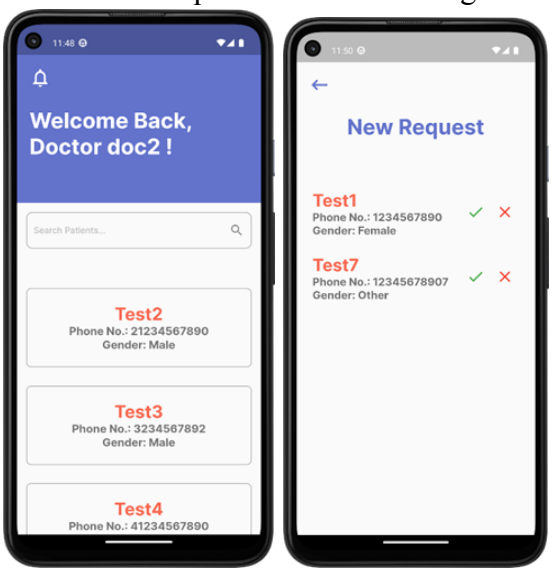
This central page serves as a navigation hub, enabling users to access various app sections. It includes forms for logging daily information. The profile section displays the patient's acceptance status and offers the option to switch doctors if needed.



6.2.1.3 Home and Profile page

Doctor's Home Screen

These are the pages which the doctor see's when they enter the app. The pages show the patients under the doctor their data and the new requests the doctor has got.

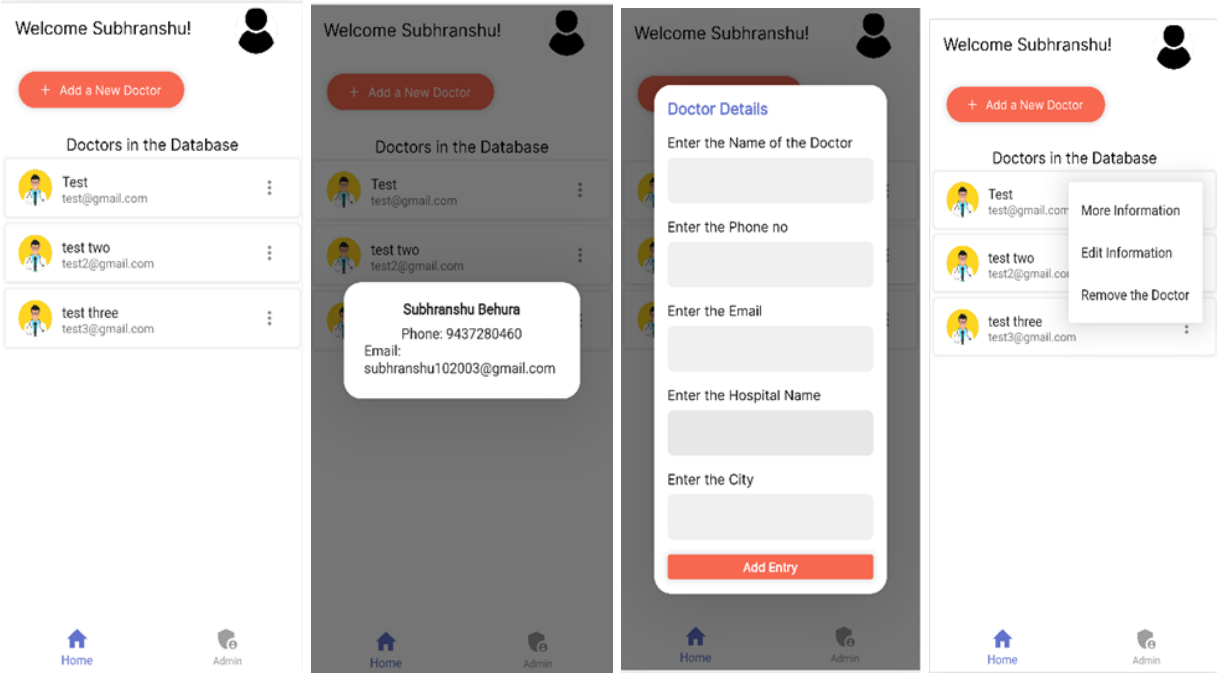


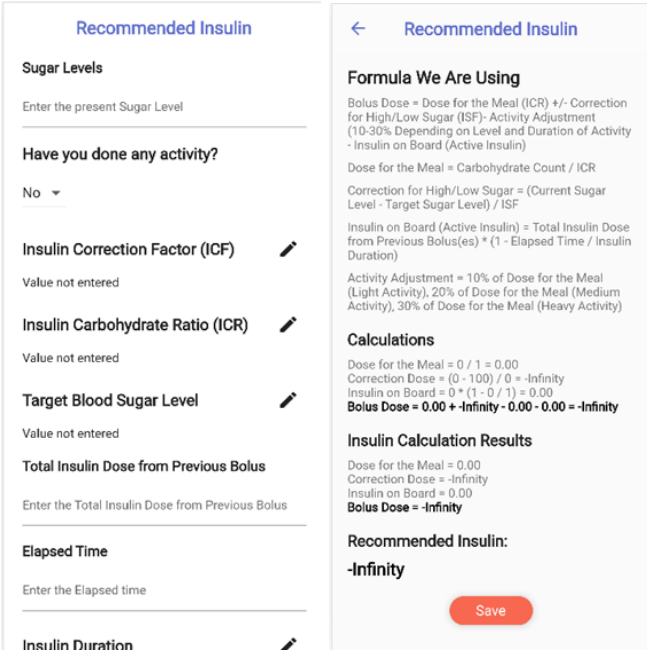
Data Visualization

These are the tools which would be used to visualize the patient's data to the doctor.

Admin Pages

These are the admin pages of the app.



Insulin Calculator

These pages would show the calculated insulin which is required for that meal.